AI ~~Chef Series~~ School #2

Let's cook a neural network!

Check-in Code: ai2school

acmurl.com/ai-school-2-slides

ACM AI

ai2school

check-in

acmucsd.com

Check-in Code: ai2school

acmurl.com/ai-school-2-slides

ACM AI

AS Event Funding Form

Please fill out this form!

Select **ACM** for the event.

bit.ly/ASFundedEvent

Check-in Code: ai2school

acmurl.com/ai-school-2-slides

ACM AI

Agenda

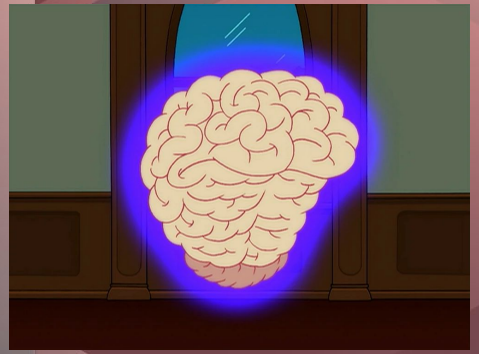
1. **Ingredients of Deep Learning**
   Gradient Descent, Backpropagation, Neural Net
2. **Recap**
   Data Preprocessing
3. **How to 🔍 a neural network?**
   Introduction to PyTorch
4. **Let's get cooking**
   Dive into colab on PyTorch
5. **Workshop project**
   Image classification on USPS Dataset

Check-in Code: ai2school

acmurl.com/ai-school-2-slides

ACM AI

Big Brain Moments: Neural Network Theory



Check-in Code: ai2school

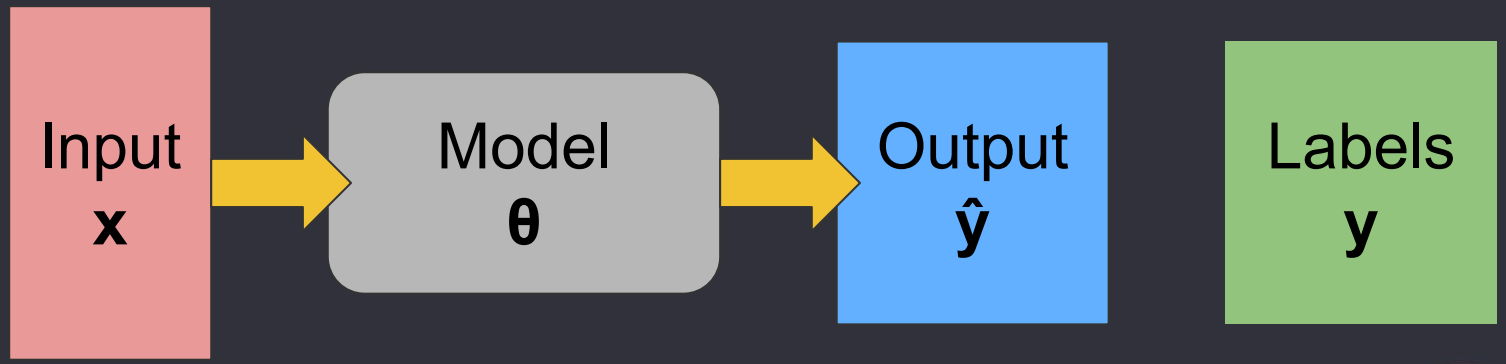
acmurl.com/ai-school-2-slides

ACM AI

General Idea Behind Machine Learning

1. Training data = (input, label)
2. Model with *learnable* parameters
3. Loss function L(ŷ,y)

$$x*\theta = y$$



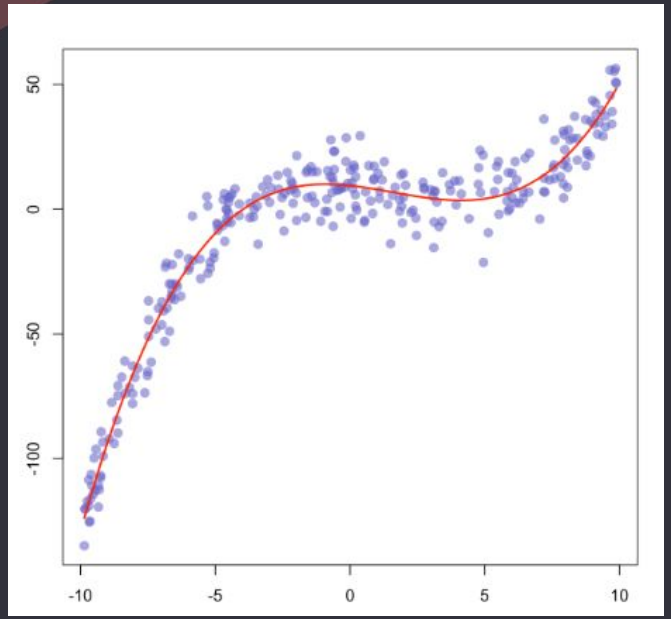
Check-in Code: ai2school

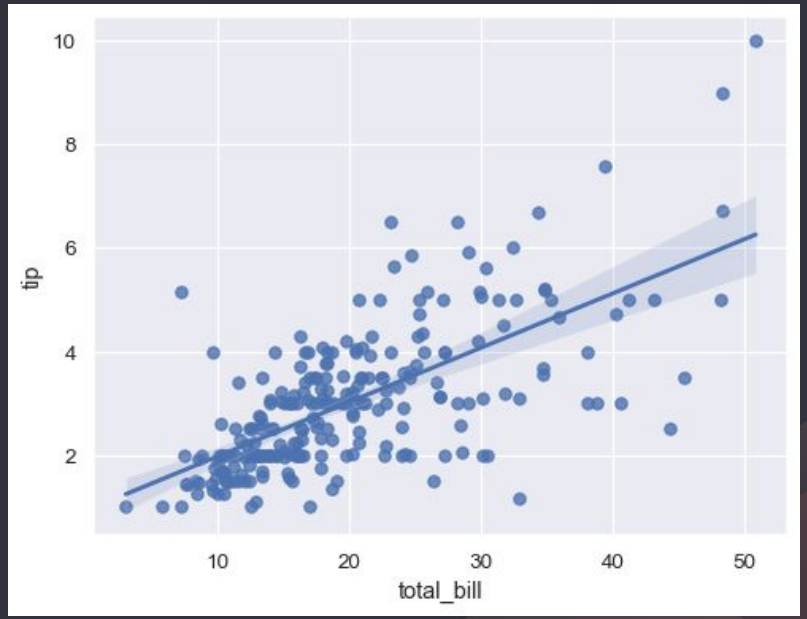
acmurl.com/ai-school-2-slides

ACM AI

Visual intuition

$x*\theta = y$





Check-in Code: ai2school

acmurl.com/ai-school-2-slides

ACM AI

Loss functions

Categorical Cross-Entropy Loss

$$CE\ Loss = -\frac{1}{n}\sum_{i=1}^{N}\sum_{j=1}^{M} y_{ij} \cdot log(p_{ij})$$

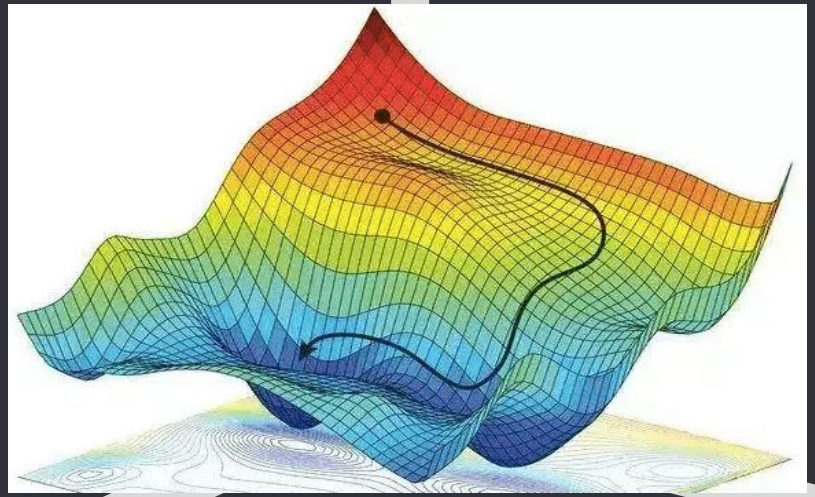
Binary Cross-Entropy/Log Loss

$$CE\ Loss = \frac{1}{n}\sum_{i=1}^{N} -\ (y_i \cdot log(p_i) + (1 - y_i) \cdot log(1 - p_i)$$

Gradient Descent (visual intuition)

L(m,b)

1. x*m + b = y_pred
2. Calculate loss with y and y_pred
3. Based on gradient (like derivative for multivariable functions) adjust parameters



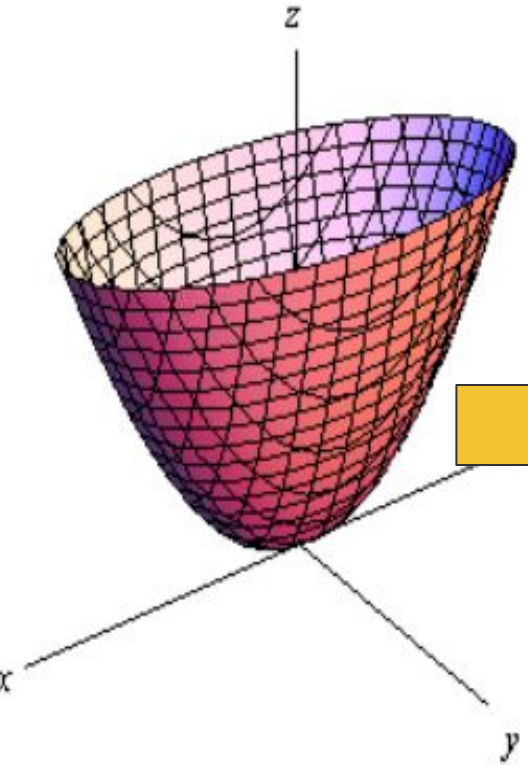
b

m

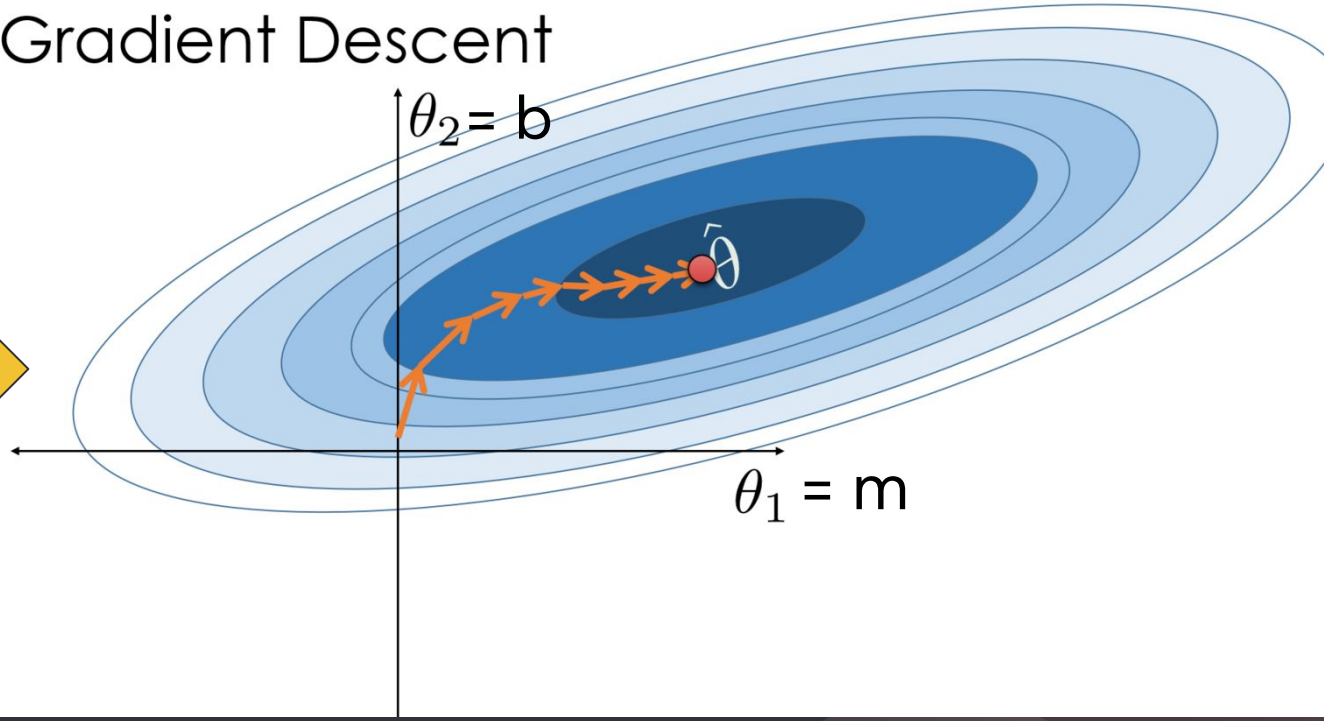
Check-in Code: ai2school

acmurl.com/ai-school-2-slides

ACM AI



Gradient Descent



Gradient Descent (algorithm)

1. Take derivative of loss w.r.t. parameters
    - ie. gradients
2. Move in the opposite direction of the gradient
3. Repeat.

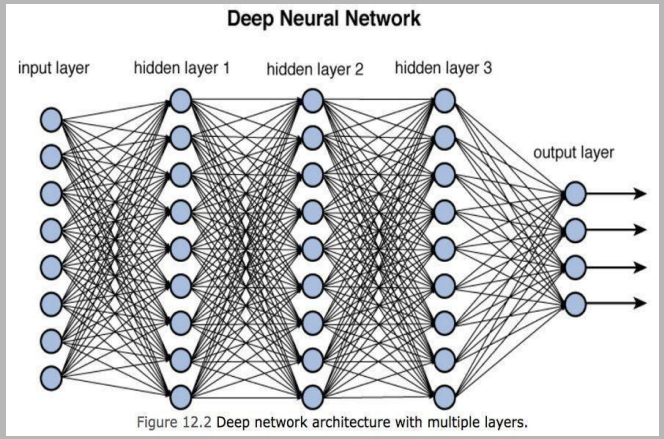
Gradient Descent Review

- ML: Train model on data instead of hand-coded algorithm
- Training == minimizing a loss function
  - Can be interpreted as loss landscape
  - Loss as function of parameters
- One way to find minima is gradient descent
  - aka ball rolling down loss landscape

Neural Net

Input
$x \in R$

Model



θ = {weights,biases}

Output
$\hat{y} \in R$

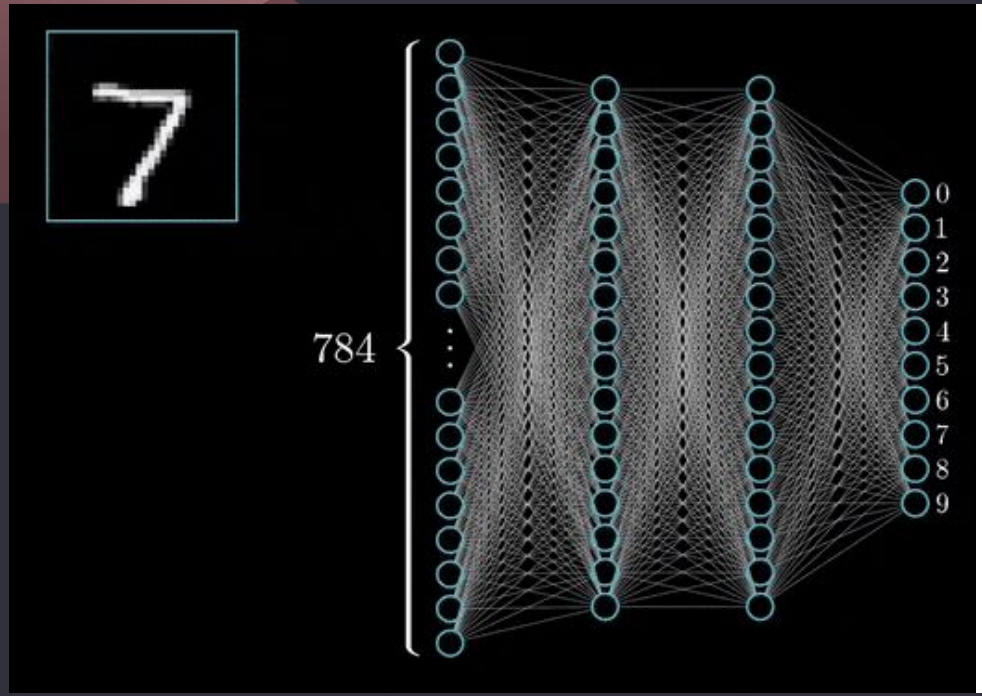
Labels
$y \in R$

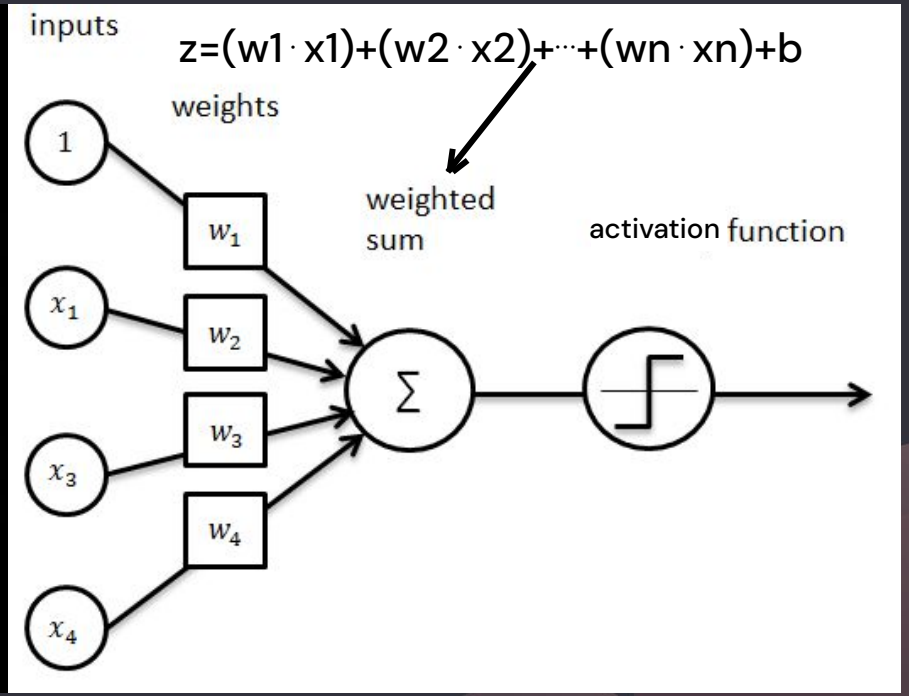
Check-in Code: ai2school

acmurl.com/ai-school-2-slides

ACM AI

Neural Net





Check-in Code: ai2school

acmurl.com/ai-school-2-slides

ACM AI

Layer Parts

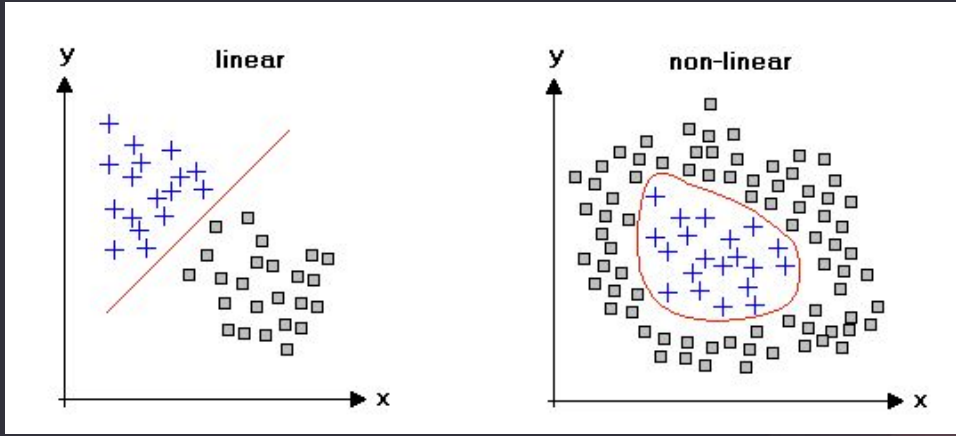
1. Input vector $\mathbf{x}^{(k)}$
2. Weights matrix $\mathbf{W}^{(k+1)}$ (learnable parameters)
3. Biases vector $\mathbf{b}^{(k+1)}$ (learnable parameters)
4. Activation function $\mathbf{g}$
5. Output vector $\mathbf{x}^{(k+1)}$

==> $$\boxed{\mathbf{x}^{(k+1)} = \mathbf{g}(\mathbf{W}^{(k+1)} * \mathbf{x}^{(k)} + \mathbf{b}^{(k+1)})}$$

Activation Function

$$x' = g(Wx + b)$$

The purpose of an activation function is to introduce non-linearity into the model, allowing the network to learn and represent complex patterns in the data.

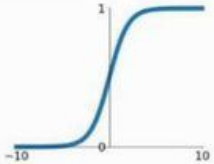


Activation Function

- Need nonlinear activation function **g**:

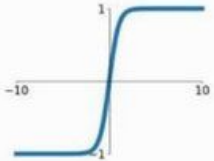
Sigmoid

$\sigma(x) = \frac{1}{1+e^{-x}}$



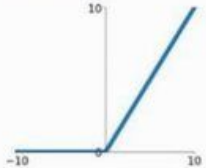
tanh

$\tanh(x)$



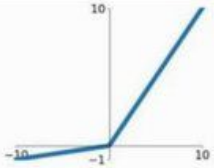
ReLU

$\max(0, x)$



Leaky ReLU

$\max(0.1x, x)$

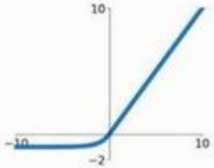


Maxout

$\max(w_1^T x + b_1, w_2^T x + b_2)$

ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



Neural Net Playground

Try creating a model that will predict classify the values correctly:

"In the output layer, the dots are colored orange or blue depending on their original values. The background color shows what the network is predicting for a particular area"

https://playground.tensorflow.org/

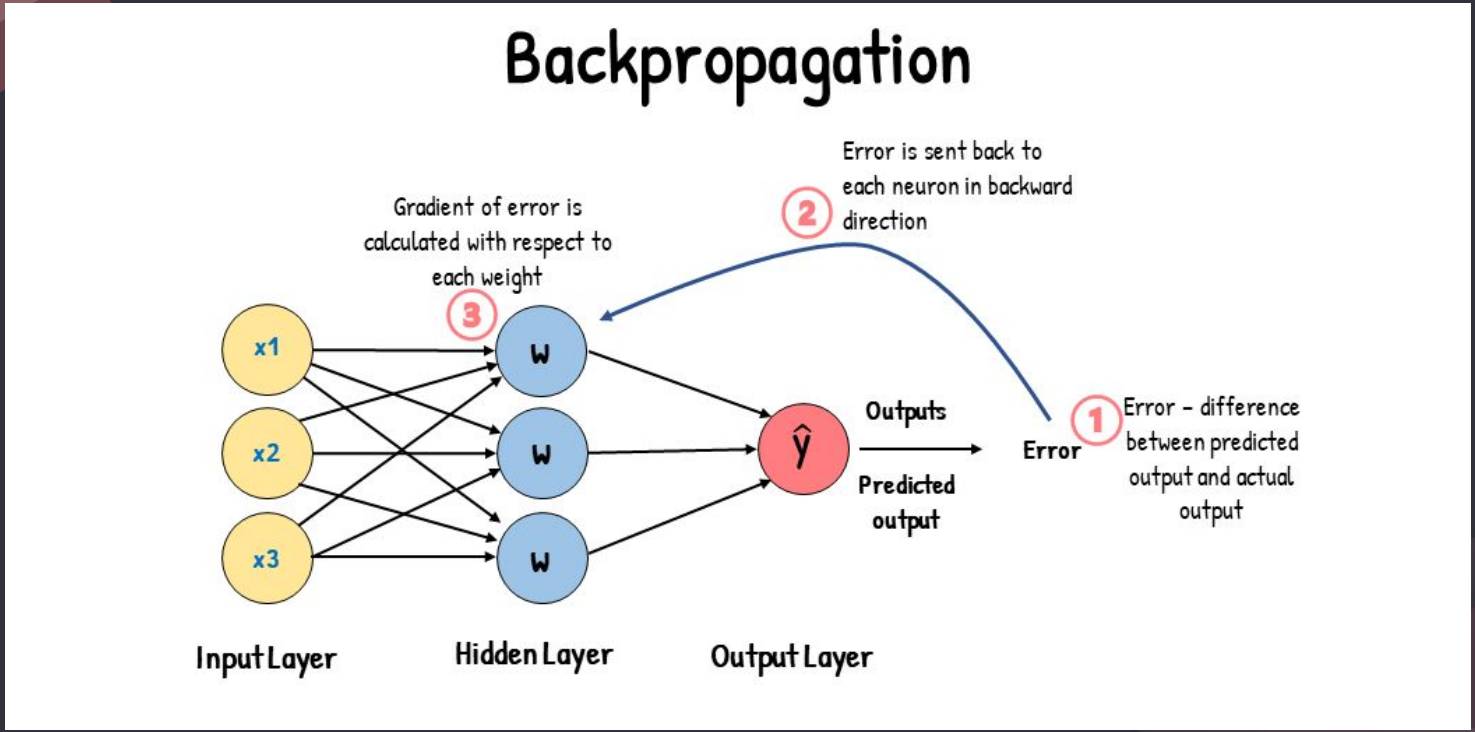
Backpropagation

How do we update our neural network weights to minimize the loss function?

Gradient descent! But to do that, we need to find the gradients of weights with respect to the loss function.

Backpropagation does this efficiently by applying the Chain Rule from calculus.

Backpropagation Idea



Check-in Code: ai2school

acmurl.com/ai-school-2-slides

ACM AI

Key takeaway from Backpropagation

1. Forward Propagation

   - Input data is passed through the network's layers to generate an output prediction.

2. Compute Loss

   - Compare output prediction with ground truth and provide the network a metric to improve.

3. Chain Rule

   - Enables faster calculation of the partial derivatives by breaking it down into smaller derivatives.

So how can we 🍳 a neural network?

Check-in Code: ai2school

acmurl.com/ai-school-2-slides

ACM AI

Recap

Check-in Code: ai2school

acmurl.com/ai-school-2-slides

ACM AI

what we'll cover

- define transformations
- apply transformations to the dataset

Your dataset

Last Time

what we'll cover 🍱

- define transformations
- apply transformations to the dataset

Your dataset

Group Activity

Discuss with the person next to you what you think a tensor is. (specifically for neural network)

Feel free to use Google.

Check-in Code: ai2school

acmurl.com/ai-school-2-slides

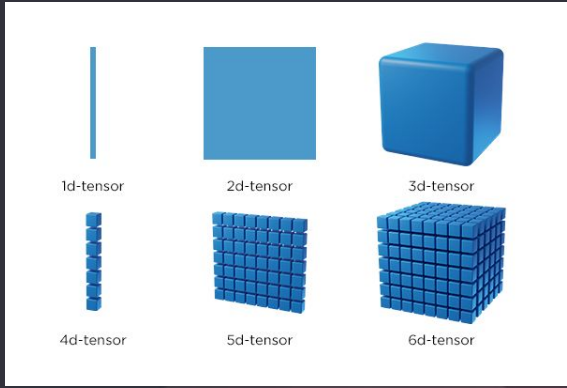
ACM AI

What are Tensors?

Multi-dimensional arrays, generalized from matrices

Tensors can hold different types of information that can be useful in a neural network

rank	type	example
0	scalar	[1]
1	vector	[1,1]
2	matrix	[[1,1], [1,1]]
3	3 tensor	[[[1,1], [1,1]], [[1,1], [1,1]]]
n	N tensor	



Check-in Code: ai2school

acmurl.com/ai-school-2-slides

ACM AI

Colab Notebook

Open our Colab Notebook

acmurl.com/aischool2-colab

Check-in Code: ai2school

acmurl.com/ai-school-2-slides

ACM AI

What kind of data can Tensors store?

- Tensors hold input data like images, text, or any structured data fed into the network
- **Weights** and **biases** in each layer of a neural network are stored as tensors
- During training, tensors store **gradients**, which represent the partial derivatives of the loss with respect to each weight and bias in the network.

Why are Tensors Important?

- Tensors are the foundational data structure for deep learning frameworks
- They allow for efficient storage and manipulation of multi-dimensional data (e.g., images, videos).
- Enable seamless transitions between CPU and GPU, making it easier to leverage hardware acceleration.

Ramping up

Build a model with Pytorch 🌶️

Check-in Code: ai2school

acmurl.com/ai-school-2-slides

ACM AI

Let's learn nn.module

Task: Create a neural network module with PyTorch that trains on Google Colab's TPU and learns to classify handwritten digits using USPS dataset.

1. Import modules
2. Specify the computing device you will be using (CPU or GPU)
3. Build your neural network architecture with nn.Module
4. Train on the USPS dataset (with the cohort).
5. Test it with the test set.
6. Try do the same thing with other datasets you explored around last time!

Check-in Code: ai2school

acmurl.com/ai-school-2-slides

ACM AI

Import modules

Use this block as where you import most of the Python modules/libraries.

Why? So we know what libraries we imported every time. Helps with your code's readability for both yourself and other people.

https://peps.python.org/pep-0008/#imports

Building model

- To train a model with gradient descent, we need to specify:
  - Forward pass: f(x)
    - Use PyTorch **nn** library
  - Backward pass: f'(x)
    - PyTorch automatically builds a "computational graph" with all the gradients in a model

- nn: PyTorch comes with lots of building blocks a.k.a. modules
  - nn.Linear, f(x) = W*x + b
  - nn.ReLU, f(x) = max(0,x)
  - Etc.
- Construct model from modules. The whole model is also a module.

Custom nn.Module

- For more custom/complicated models, we subclass nn.Module and implement:
  - **__init__()** method
    - Initialize relevant modules (nn.Linear, nn.ReLU, nn.Sequential(), etc.)
  - **forward()** method
    - Perform operations on the input, using components created in __init__()

- More flexible than nn.Sequential(), which applies provided nn.Module's in a strict sequential order
- Can also be used to create custom building blocks

Custom nn.Module (example)

```
class Net(nn.Module):
    def __init__(self, input_size, hidden_sizes, num_classes):
        super(Net, self).__init__()
        self.input_layer = nn.Linear(input_size, hidden_sizes[0])
        self.hidden_layers = nn.ModuleList([
            nn.Linear(hidden_sizes[i], hidden_sizes[i+1]) for i in ra
        ])
        self.output_layer = nn.Linear(hidden_sizes[-1], num_classes)

    def forward(self, x):
        out = nn.ReLU()(self.input_layer(x))
        for hidden_layer in self.hidden_layers:
            out = nn.ReLU()(hidden_layer(out))
        out = self.output_layer(out)
        return out
```

Understanding `torch.device`

torch.device: Represents the device where a torch.Tensor is or will be allocated.

Common: "cpu", "cuda"
Others: "mps", "xpu", "xla", "meta"

FAQ: Can we have PyTorch operations on multiple devices? E.g., one on CPU and the other on GPU?

Understanding `torch.device`

torch.device: Represents the device where a torch.Tensor is or will be allocated.

Common: "cpu", "cuda"
Others: "mps", "xpu", "xla", "meta"

FAQ: Can we have PyTorch operations on multiple devices? E.g., one on CPU and the other on GPU?

Yes but when doing operations, the elements should be on the same device.

Load Datasets

For learning purpose, we'll first both work on the USPS dataset together.

Paste the dataset loading code from our last class!

Check-in Code: ai2school

acmurl.com/ai-school-2-slides

ACM AI

Train Your Model

1. Define your neural network module
2. Initialize the loss function
3. Read the train_loop
4. Read the test_loop
5. Run the last cell to train and test your model

Check-in Code: ai2school

acmurl.com/ai-school-2-slides

ACM AI

Make the model perform better 🧠

OK so how can we make the model perform better? And what did I mean by "better"?

For the purpose of this lesson, let's make the final accuracy on test set the metric for a better model. The higher the accuracy, the better it is.

There are many ways to make your neural network model better.

Here are a few pointers:

- Read this blog post from the goat Andrej Karpathy on recipes for training neural networks.
- Experiment with different optimizers. Take a look at `torch.optim`
- Try using Learning Rate Scheduling like `torch.optim.lr_scheduler.StepLR`. For how you can adjust learning rate, see here.
- Increase the number of epochs
- Add regularization. Try using drop out layers.
- More data augmentation. (Pop quiz, how do you do that? by editing what?)

Earn Points by Providing Feedback on the ACM Portal!

Your insights help us improve and grow our ACM community.

Check-in Code: ai2school

acmurl.com/ai-school-2-slides

ACM AI

Thank You

Do you have any questions?

contact@acmucsd.org

acmurl.com/**discord**

acmurl.com/**instagram**

acmurl.com/**youtube**

Check-in Code: ai2school

acmurl.com/ai-school-2-slides

ACM AI